

中小计划排级列生成算法说明

medium_small_raw 实现

2026 年 6 月 5 日

目录

1 业务定位	3
2 运行入口与输出	3
2.1 联合生成中计划和小计划	3
2.2 给定外部中计划后生成小计划	4
3 输入口径	4
3.1 大计划	4
3.2 堆场快照	4
3.3 箱区功能、船期和泊位距离	5
4 需求组	5
4.1 粗属性组	5
4.2 细属性组	5
4.3 预测兜底组	5
5 排级列	5
6 四阶段业务流程	6
7 主问题和硬约束	7
8 列生成、diving 和 repair	7
9 目标函数	8
9.1 细属性组集中	8
9.2 粗属性组集中	8
9.3 大计划继承	9

目录	2
9.4 泊位距离和作业窗口	9
10 排级可行性的含义	9
11 中计划和小计划的一致性	9
12 诊断字段	10
13 主要参数	10

1 业务定位

`medium_small_raw` 是一套中小计划排级列生成程序。它在大计划给出的航次-流向-箱区-尺寸配额基础上，生成两类结果：

- `medium_plan.csv`：中计划输出，按粗属性组在箱区/贝位上的计划量汇总。
- `small_plan.csv`：小计划输出，按细属性组落到具体箱区、贝位和排号。

这套程序的核心目标有三项：

- 细属性组尽量集中，减少同一细属性组跨箱区、跨 6 小贝区块和跨贝位。
- 粗属性组尽量箱区集中。小数量粗属性组倾向集中在一个箱区；大数量粗属性组允许使用多个箱区，但避免很小的尾巴箱区，并保持主要箱区之间相对均衡。
- 尽量继承大计划。箱量优先落在大计划给定的同航次、同流向、同尺寸箱区内；必要时按阶段逐步放宽箱区范围。

排级约束是本目录与贝位级列生成版本的主要区别。每个被选择的列都带有 `row_allocation`，小计划输出包含 `row_no`。模型不仅控制贝位总容量，还控制每个贝位每一排的物理容量、尺寸容量和不混排属性。默认排级不混属性为卸港 `port`，默认贝位级不混属性为尺寸 `size` 和箱高 `height`；这些属性可以通过输入中的 `attribute_rules` 覆盖。

2 运行入口与输出

2.1 联合生成中计划和小计划

联合生成入口为：

```
.\.venv\Scripts\python.exe medium_small_raw\run_column_generation_planner.py
```

默认数据集为 0519。数据目录、大计划文件、目标航次、规划时刻和规划窗口都可以通过命令行参数覆盖。输出目录包含：

- `small_plan.csv`：排级小计划。
- `medium_plan.csv`：由被选列汇总得到的中计划。
- `small_plan_six_bay_blocks.csv`：按连续 6 小贝区块汇总的小计划。
- `big_plan_used.csv`：本次使用的大计划配额行。
- `generated_columns.csv`：本次生成过的候选列。
- `diagnostics.json`：需求、列池、阶段、未放置箱、集中性、继承性和运行时间诊断。

2.2 给定外部中计划后生成小计划

外部中计划入口为：

```
.\.venv\Scripts\python.exe medium_small_raw\run_small_plan_from_medium.py --medium-plan
path\to\medium_plan.csv
```

该入口固定使用 doc-only 口径：只规划当前未进场资料箱，不生成预测兜底组。外部中计划形成两类硬上界：

- 粗属性组-箱区上界 (v, f, p, s, a) 。
- 若外部中计划带有贝位字段，则形成粗属性组-箱区-贝位上界 (v, f, p, s, a, b) 。

小计划在这些上界、排级容量和不混排规则内落箱。若外部中计划给出的箱区/贝位本身无法容纳某些资料箱，未放置箱量会写入 diagnostics.json。

3 输入口径

3.1 大计划

大计划提供航次、流向、箱区、尺寸和计划箱量。当前读取器支持常见字段别名，例如 voy_id、voyage_id、area_no、size、planned_qty、planned_boxes 等。45ft 在继承大计划配额时归入 40ft 大计划尺寸口径；在小计划落排时仍按 45ft 规则处理。

大计划用于三件事：

- 推导默认目标航次。没有显式传入 --voyages 时，程序读取规划日期上的 OF 行作为目标航次来源。
- 构造箱区配额上界 $Q_{vfa\bar{s}}$ ，其中 $\bar{s}$ 为大计划尺寸口径。
- 形成继承偏好。精确大计划箱区成本最低，偏离大计划箱区会进入 fallback 层级成本和大计划模式偏离目标。

3.2 堆场快照

堆场快照提供箱区、贝位、排号和在场箱信息。程序从中构造：

- 贝位物理容量和尺寸容量。
- 每个贝位每一排的物理容量和尺寸容量。
- 40ft/45ft 占用的连续小贝 footprint。
- 已有箱尺寸、箱高、卸港等属性。
- 当前已经在场的箱，用于从资料箱需求中剔除。

当某个贝位已有箱属性与待放需求组冲突时，该贝位或对应排不进入可行候选。排级可行性直接在列生成、整数主问题和 repair 中共同生效。

3.3 箱区功能、船期和泊位距离

箱区功能表用于判断某箱区是否支持某流向。船期和规划时刻用于计算目标航次窗口、需求比例和作业窗口。泊位距离表用于给航次-箱区组合增加距离成本。

箱区内其它装船作业也会进入目标：规划窗口内存在作业冲突会增加成本；规划窗口后 24 小时内有后续装船作业时，相关箱区会获得一定偏好。

4 需求组

4.1 粗属性组

粗属性组是中计划的业务单位，默认键为：

$$g = (v, f, p, s),$$

其中 v 为航次， f 为流向， p 为卸港， s 为真实尺寸 20、40 或 45。粗属性组箱量来自 `calculate_medium_demands`，并受大计划总配额约束。

4.2 细属性组

细属性组是小计划落排单位，默认键为：

$$h = (v, f, p, s, \eta, w, \sigma, r),$$

其中 η 为箱高， w 为重量等级， σ 为特殊积载代码， r 为预配/预留标记。资料箱读取后会先剔除当前已经在场的箱，再按细属性聚合。

4.3 预测兜底组

联合生成默认 `original` 模式下，模型内部还包含预测兜底组。预测兜底组用于补足中计划需求中尚未被资料箱覆盖的箱量，并参与容量占用和中计划输出。默认 `small_plan.csv` 只写出资料箱细属性组，预测兜底组不写入小计划。

需求模式含义如下：

- `original`：中计划按中计划需求输出，小计划只输出资料箱；预测兜底组参与模型。
- `medium`：小计划输出包含资料箱和预测兜底组，总量对齐中计划需求。
- `medium-with-doc-floor`：保留全部资料箱，并补足中计划不足部分。
- `doc-only`：只规划当前未进场资料箱，不生成预测兜底组。

5 排级列

一列表示一个细属性组在一个贝位上的一批箱量放置方案：

$$c = (h, b, q, R_c),$$

其中 h 为细属性组, b 为贝位, q 为箱量, R_c 为排级分配。对 20ft 箱, footprint 是一个小贝; 对 40ft/45ft 箱, footprint 是一对连续小贝。40ft/45ft 的 row_allocation 会同时记录 footprint 内两个小贝在同一排上的占用。

列生成时, 候选列必须满足:

- 箱区支持对应流向, 或由用户箱区规则强制允许。
- 20ft 不放箱区边贝, 45ft 只放箱区边贝。
- 40ft/45ft 必须存在可用的大贝 partner。
- 贝位已有尺寸、箱高等贝位级属性与需求组相容。
- 排已有卸港等排级属性与需求组相容。
- footprint 内所有小贝在同一排上都有足够的物理容量和尺寸容量。
- 列箱量不超过需求组剩余箱量、贝位容量、排容量和相关箱区配额。

每个候选贝位会生成若干箱量选项, 既包含最大可放量, 也包含小箱量和余数箱量, 用于在集中和可行之间取得平衡。每个列在 generated_columns.csv 中写出 row_allocation 和 stack_units。

6 四阶段业务流程

求解过程采用 “stage0 主分配 + stage1–stage3 repair 补齐 + repair LNS + 箱区内贝位重排” 的流程。stage0 和 repair 决定箱区层方案; 箱区内贝位重排在箱区层方案固定后运行, 只重新安排箱区内部的贝位和排级可行列。

阶段	业务含义	箱区范围和约束
stage0	大计划范围内的主体分配	候选箱区必须是同航次、同流向、同大计划尺寸口径下有精确大计划配额的箱区。大计划配额作为硬上界。列生成和主问题主要在这一范围内寻找继承性好、集中性好的方案。
stage1a	精确大计划配额内补箱	repair 中继续使用同 stage0 的精确大计划箱区范围, 并严格执行大计划配额上界。
stage1b	同航次流向的大计划箱区补箱	候选箱区放宽到该需求组所属航次、流向的大计划覆盖箱区, 不再用精确箱区配额卡住单个需求组。
stage2	大计划覆盖箱区补箱	候选箱区放宽到任意被大计划覆盖的箱区, 用于处理同航次流向范围内仍无法放下的箱。
stage3	功能兼容兜底补箱	候选箱区放宽到功能兼容且满足用户箱区策略、容量和排级规则的箱区。该阶段优先保证小计划可执行。

每个阶段都遵守排级容量、不混排、边贝、尺寸、箱高、外部中计划配额等硬约束。阶段放宽的是箱区候选范围和大计划精确配额, 不放宽物理和排级可行性。

stage0 后，程序先求一个“最少未放置箱”的整数模型。若该模型达到最优，则得到 stage0_unplaced_cap，后续最终主问题和 repair 都不能超过这个未放置箱上限。若 stage0 已经证明零未放置，后续阶段保持零未放置。

7 主问题和硬约束

主问题选择当前列池中的列。整数阶段每列为二进制变量 x_c ，每个细属性组有未放置变量 u_h 。需求覆盖约束为：

$$\sum_{c:h(c)=h} q_c x_c + u_h = d_h, \quad \forall h.$$

贝位、排和配额硬约束包括：

$$\begin{aligned} \sum_{c:b(c)=b} q_c x_c &\leq Cap_b, \\ \sum_{c:b(c)=b,s(c)=s} q_c x_c &\leq Cap_{bs}, \\ \sum_{c:(b,r)\in R_c} q_{cr} x_c &\leq Cap_{br}, \\ \sum_{c:(b,r,s)\in R_c} q_{cr} x_c &\leq Cap_{brs}, \\ \sum_{c:(v,f,a,\bar{s})} q_c x_c &\leq Q_{vfa\bar{s}}. \end{aligned}$$

对于同一贝位，模型用选择变量限制其尺寸、箱高等贝位级属性只能取一个值；对于同一排，模型限制卸港等排级属性只能取一个值。40ft/45ft 的 footprint 约束保证两边小贝同步占用，排级分配也保证两边小贝在对应排都有容量。

外部中计划入口还会加入：

$$\sum_{c:(v,f,p,s,a)} q_c x_c \leq E_{vfpsa},$$

以及可选的贝位级中计划上界：

$$\sum_{c:(v,f,p,s,a,b)} q_c x_c \leq E_{vfpsab}.$$

未放置变量用于表达当前硬约束下无法覆盖的剩余箱量。它不是容量或配额的替代品；容量、配额和排级规则始终是硬约束。最终方案优先比较未放置箱数，其次比较目标能量，最后比较选中列数。

8 列生成、diving 和 repair

列生成从初始列池开始，反复求解线性松弛主问题，并根据对偶价格对未进入列池的候选列定价。负检验数列优先进入列池。若常规定价停滞，程序还会加入低检验数列和整数可行性导向的 primal expansion 列，使后续整数主问题拥有更丰富的选择。

列生成结束后，程序在当前列池上执行 diving 主问题。diving 逐步固定 LP 解中的高价值列，并在遇到 LP 未放置箱时收缩固定批次或停止。diving 后还会执行若干局部 improvement 轮，释放部分细属性组做小规模重优化。

如果主问题结果仍有未放置箱，程序进入未放置修复：

- staged greedy repair 按 stage1a、stage1b、stage2、stage3 顺序补箱，并比较默认组顺序和粗属性组集中优先顺序。
- repair LNS 以当前最好解为 incumbent，每轮释放一批细属性组，在剩余容量和剩余配额内求小规模 SCIP 子问题。
- 粗属性组 compaction LNS 默认关闭，打开后用于额外压实粗属性组箱区分布。

repair 候选方案按如下顺序排序：

(未放置箱数, 目标能量, 选中列数).

因此 repair LNS 只有在未放置箱数更少，或未放置箱数相同但目标更好时才替换 incumbent。零未放置 incumbent 下，LNS 子问题保持零未放置，不通过多留箱来换集中性。

repair 结束后，程序执行箱区内贝位重排。该步骤固定每个细属性组在每个箱区的箱量 ($group_id, area_no$)，不改变箱区层分配、不改变大计划继承比例，也不改变每个粗属性组在每个箱区的合计箱量。重排只在同一箱区内重新选择贝位，并为所选贝位生成排级可行的 row_allocation。排号在这里主要用于证明贝位方案能够真实落排，重排评分只关心细属性组贝位集中、粗属性组箱区内贝位集中和小尾巴贝位减少。

箱区内贝位重排逐箱区尝试。某个箱区若无法在排级硬约束下完成重排，则该箱区保留 repair 后的原贝位方案；其它箱区仍可继续尝试。只有当整体贝位集中性评分变好，且所有 ($group_id, area_no$) 箱量完全一致时，重排方案才会成为最终方案。若外部中计划给定贝位级硬配额，箱区内贝位重排不运行，因为贝位层方案已经由外部输入固定。

9 目标函数

9.1 细属性组集中

同一细属性组跨箱区、跨 6 小贝区块、跨贝位都会产生惩罚。对应参数为：

- --fine-group-area-penalty, 默认 80。
- --fine-group-block-penalty, 默认 35。
- --fine-group-bay-penalty, 默认 8。

9.2 粗属性组集中

同一粗属性组在同一箱区内使用更多 6 小贝区块和更多贝位会产生惩罚，参数为 --coarse-area-block-penalty 和 --coarse-area-bay-penalty。

粗属性组箱区目标分两类：

- 小数量粗属性组：需求量不超过 --medium-concentrated-group-threshold, 默认 26。目标强烈惩罚跨箱区，并奖励最大箱区承载更多箱量。

- 大数量粗属性组:不强求只用极少箱区,而是惩罚低于 `--medium-large-group-min-area-boxes` 的小尾巴箱区, 默认阈值 10; 同时用均衡项避免主要箱区之间差异过大。

大数量粗属性组的相关参数包括 `--medium-large-group-small-area-penalty`、`--medium-large-group-area-open-penalty` 和 `--medium-large-group-target-area-boxes`。

箱区内贝位重排阶段继续使用“细属性组少跨贝位、粗属性组少跨贝位、减少小尾巴贝位”的集中性口径。该阶段不再考虑大计划偏离、泊位距离、作业窗口等目标, 因为箱区层方案已经固定; 它只在不改变箱区量且满足排级可行性的前提下改善贝位层分布。

9.3 大计划继承

大计划继承由三部分表达:

- 精确大计划箱区配额上界。`stage0` 和 `stage1a` 严格执行同航次、同流向、同大计划尺寸口径的箱区配额。
- `--required-area-reward`。列落在大计划要求箱区时获得基础成本奖励, 默认 1000。
- `--big-plan-fallback-tier-penalty` 和 `--big-plan-area-deviation-penalty`。前者按候选箱区偏离大计划的层级增加成本; 后者惩罚航次-流向-大计划尺寸合计分布相对大计划比例的偏离。

大计划偏离目标约束的是合计箱区模式, 不要求每个粗属性组都机械地按大计划比例拆分。粗属性组集中目标和大计划继承目标共同决定最终箱区选择。

9.4 泊位距离和作业窗口

航次-箱区使用会带来泊位距离成本。规划窗口内已有其它装船作业的箱区成本更高; 规划窗口后 24 小时内有后续装船作业的箱区会有一定偏好。`stage3` 选择大计划之外的功能兜底箱区时, 这类成本有助于把箱放在更合理的作业位置。

10 排级可行性的含义

排级可行性表示: 对 `small_plan.csv` 中每一行, 输出的 `area_no`、`bay_no`、`row_no` 和 `planned_boxes` 能映射回列的 `row_allocation`; 同一贝位同一排上的箱量不超过该排物理容量和尺寸容量; 同一排不会出现不允许混放的属性组合。

对 40ft/45ft 箱, `row_allocation` 记录 `footprint` 内两个小贝的排级占用。只有当两个小贝在同一排上都满足容量和属性相容时, 该列才是可行列。这样可以避免“贝位层面看似可行, 落到排后无法摆放”的情况。

11 中计划和小计划的一致性

联合生成时, `medium_plan.csv` 直接由被选列按粗属性组、箱区和贝位汇总得到。每一行包含:

plan_level, voyage_id, flow, port, size, area_no, bay_no, six_bay_block_id, six_bay_block_bays, planned_boxes, document_boxes, forecast_fallback_boxes

small_plan.csv 则进一步落到排, 包含:

plan_level, voyage_id, group_id, demand_source, flow, port, size, height, weight_class, special_stow, special_stow_code, area_no, bay_no, row_no, row_allocation, six_bay_block_id, six_bay_block_bays, six_bay_block_total_boxes, planned_boxes

默认 original 口径下, 小计划只写资料箱, 预测兜底组只体现在中计划的 forecast_fallback_boxes 中。诊断文件中的 small_medium_consistency_violations 和 small_medium_bay_consistency_violations 用于检查小计划汇总是否超过中计划对应口径。

12 诊断字段

diagnostics.json 是判断结果是否正常的主要文件。常用字段包括:

- planned_medium_boxes、planned_small_boxes、unplaced_boxes。
- stage0_unplaced_cap 和 stage0_unplaced_cap_source。
- master_status、master_algorithm、final_column_count、selected_column_count。
- repair_lns_rounds_run、repair_lns_improvements、repair_lns_stop_reason。
- post_repair_area_relayout_enabled、post_repair_area_relayout_accepted、post_repair_area_relayout_before、post_repair_area_relayout_after, 记录箱区内贝位重排是否运行、是否接受以及重排前后的贝位集中性指标。
- medium_big_plan_inheritance.inheritance_ratio。
- medium_fragmentation 下的 tiny_area_rows、small_coarse_multi_area_groups、large_coarse_tiny_area_rows。
- attribute_rules, 记录本次使用的粗属性、细属性、贝位不混属性和排不混属性。

这些字段共同解释三类问题: 是否所有箱都放下、粗细属性组集中性是否合理、大计划继承性是否保持在可接受范围内。

13 主要参数

参数、默认值与业务含义

--max-iterations=30: 列生成最大迭代次数。

--columns-per-iteration=2500: 每轮最多加入的负检验数列数量。

--initial-columns-per-group=16: 每个细属性组初始候选贝位数量。

--max-candidate-bays-per-group=500: 每个细属性组最多保留的候选贝位数量。

--mip-time-limit=120、--mip-gap=0.01: 最终主问题求解限制。

--diving-improvement-rounds=6、--diving-improvement-time-limit=6.0: diving 后局部重优化轮数和单轮时间。

--repair-lns-rounds=2、--repair-lns-time-limit=3.0: 未放置修复 LNS 轮数和单轮时间。

--coarse-compaction-lns-rounds=0: 额外粗属性组压实 LNS, 默认关闭。

post_repair_area_relayout_enabled=True: repair 后箱区内贝位重排, 固定每个细属性组在每个箱区的箱量, 只优化箱区内贝位集中性。

post_repair_area_relayout_max_patterns=1: 箱区内贝位重排时每个候选贝位只取一个排级可行模式, 排只作为可行性证明, 不作为主要优化目标。

--medium-concentrated-group-threshold=26: 小数量粗属性组和大数量粗属性组的分界。

--medium-small-group-area-split-penalty=2400: 小数量粗属性组跨箱区惩罚。

--medium-small-group-fragment-penalty=90: 小数量粗属性组碎片惩罚。

--medium-large-group-min-area-boxes=10: 大数量粗属性组小尾巴箱区阈值。

--medium-large-group-small-area-penalty=900: 大数量粗属性组小尾巴箱区惩罚。

--medium-large-group-target-area-boxes=60: 大数量粗属性组目标箱区数量的参考箱量。

--unplaced-penalty=100000: 未放置变量目标权重; 未放置控制主要由 stage0 上限和 repair 不恶化约束保证。

--required-area-reward=1000: 落在大计划要求箱区的奖励。

--big-plan-area-deviation-penalty=3: 大计划合计箱区模式偏离惩罚。

--big-plan-fallback-tier-penalty=20: 偏离大计划候选层级的惩罚。

--full-column-pool: 直接枚举当前可枚举列池, 跳过常规定价循环。

--no-scip: 使用确定性贪心回退, 适合检查数据链路, 不代表正式优化质量。
